

MODULE 1

Maths pour le Deep Learning

Doctorant	Tidiane DIALLO
Directeur	Pr. Abdoul Aziz Ciss — EPT
Période	Semaines 1-2 Avril 2026
Niveau	Débutant → Bases solides
Durée estimée	6-8 heures

Objectifs du module

- Comprendre les vecteurs et matrices (briques du DNN)
- Maîtriser la dérivée et le gradient (base de l'apprentissage)
- Comprendre la descente de gradient (comment le réseau apprend)
- Calculer une fonction de perte (comment mesurer l'erreur)
- Savoir faire ces calculs en Python/NumPy

Chapitre 1 — Vecteurs & Matrices

1.1 Vecteur

Un vecteur est une liste ordonnée de nombres. Dans un DNN, un vecteur représente les valeurs d'entrée (ex: les pixels d'une image aplatie).

Notation: $\mathbf{x} = [x_1, x_2, \dots, x_n]$

```
import numpy as np

# Vecteur = 1 image de 3 pixels
x = np.array([0.5, 0.8, 0.2])
print("Taille:", x.shape) # (3,)
print("2ème valeur:", x[1]) # 0.8
```

1.2 Matrice et Multiplication matricielle

Une matrice est un tableau 2D de nombres. Dans un DNN, la matrice W contient les poids qui transforment les entrées en activations. C'est l'opération la plus répétée dans un réseau de neurones.

$$\mathbf{z} = \mathbf{W} \cdot \mathbf{x} + \mathbf{b}$$

Où W = matrice des poids, x = vecteur d'entrée, b = vecteur de biais, z = vecteur de sortie (avant activation).

```
W = np.array([[1, 0, 2], # matrice 2x3
              [3, 1, 0]])
x = np.array([1, 2, 3]) # vecteur 3x1

z = np.dot(W, x) # produit matriciel
print(z) # [7, 5]

# Vérification à la main:
# z[0] = 1x1 + 0x2 + 2x3 = 7
# z[1] = 3x1 + 1x2 + 0x3 = 5
```

Lien avec ta thèse

Chaque couche d'un DNN effectue: $z = W \cdot x + b$. Voler un réseau de neurones = retrouver les matrices W et les biais b de chaque couche. C'est exactement l'objectif d'une attaque d'extraction!

Chapitre 2 — Dérivées & Gradient

2.1 Dérivée d'une fonction

La dérivée d'une fonction en un point mesure son taux de variation: est-ce que la fonction monte ou descend, et à quelle vitesse?

Fonction $f(x)$	Dérivée $f'(x)$	Utilisée pour
x^n	$n \cdot x^{n-1}$	Polynômes
e^x	e^x	Activation exponentielle
$\ln(x)$	$1/x$	Log-vraisemblance
$\text{sigmoid}(x)$	$\text{sigmoid}(x) \cdot (1 - \text{sigmoid}(x))$	Activation sigmoïde
ReLU: $\max(0, x)$	0 si $x < 0$, 1 si $x > 0$	Activation ReLU (thèse!)

2.2 Gradient (dérivée multivariable)

Le gradient ∇f est un vecteur qui contient les dérivées partielles par rapport à chaque variable. Il pointe dans la direction de la montée maximale. En DNN, on calcule le gradient de la fonction de perte par rapport à chaque poids.

$$\nabla f(x, y) = [\partial f / \partial x, \partial f / \partial y]$$

2.3 Descente de gradient

La descente de gradient est l'algorithme central d'entraînement d'un DNN. On met à jour les poids dans la direction OPPOSÉE au gradient pour réduire la perte.

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \nabla L(\mathbf{w})$$

α (alpha) = learning rate = taille du pas (valeur typique: 0.001 à 0.01)

```
# Descente de gradient simple
def f(x): return x**2 + 2*x + 1 # (x+1)²
def df(x): return 2*x + 2 # dérivée

x = 5.0 # point de départ
lr = 0.1 # learning rate

for i in range(20):
    x = x - lr * df(x) # ← formule de mise à jour

print(f"Minimum trouvé: x={x:.4f}") # → x ≈ -1.0
```

Chapitre 3 — Fonctions de Perte

La fonction de perte (loss) mesure à quel point le réseau se trompe. L'entraînement consiste à minimiser cette perte.

Nom	Formule	Utilisation
MSE (régression)	$L = (1/n) \cdot \sum (y - \hat{y})^2$	Prédire des valeurs réelles
Cross-Entropy	$L = -\sum y \cdot \log(\hat{y})$	Classification multi-classes
Binary Cross-Entropy	$L = -(y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y}))$	Classification binaire

Exercices Pratiques

Exercice 1.1 — Matrices [30min]

Créer une matrice W (3×4) et un vecteur x (4×1) en NumPy. Calculer $z = W \cdot x$. Vérifier le résultat à la main pour la première ligne.

Exercice 1.2 — Gradient visuel [45min]

Implémenter la descente de gradient sur $f(x) = x^3 - 4x^2$. Visualiser avec matplotlib. Trouver les deux minima locaux.

Exercice 1.3 — Backprop à la main [1h]

Calculer à la main les gradients d'un réseau $2 \rightarrow 3 \rightarrow 1$ avec activation sigmoid. Comparer avec le calcul automatique de PyTorch (autograd).

Ressources Recommandées

Type	Ressource	Durée	Lien
Vidéo	3Blue1Brown: Essence of Linear Algebra	3h	youtube.com/watch?v=fNk_zzaMoSs
Vidéo	3Blue1Brown: Neural Networks	1h	youtube.com/playlist/PLZHQOb...
Livre	Deep Learning - Goodfellow (gratuit)	Réf.	deeplearningbook.org
Code	NumPy Tutorial officiel	2h	numpy.org/learn
Code	Karpathy: micrograd (backprop from scratch)	2h30	youtube.com/watch?v=VMj-3S1tku0